학습 노트: 프롬프트 엔지니어링 5기법 (Few-shot / CoT / RAG 주입 / Structured Output / Domain Rules)

작성일: 2026-05-24 최종 수정: 2026-05-26 관련 파일: `dev2/prompts/severity_judge.py`, `dev2/schema.py`, `dev2/vector_store/` 복습 예정: D+1, D+3, D+7 관련 **Task**: Task 4-b — LLM severity 재판단 프롬프트 빌더 구현 + `/feature-dev` Issue 스키마 확장

한 줄 요약

CLI 도구(rule-based)가 기계적으로 매긴 severity를, LLM이 "예시 + 단계적 사고 + 과거 사례 + 정해진 답안 형식 + 도메인별 규칙"의 5가지 장치로 더 똑똑하게 다시 판단하도록 만드는 프롬프트 작성 기술.

큰 그림: 왜 LLM이 다시 판단해야 하나?

Task 3에서 한 것 **(rule-based)**

`dev2/tools/python/pylint_tool.py` 같은 CLI 도구는 고정된 표로 severity를 매긴다.

```
{"error": "Critical", "warning": "Major", "convention": "Minor"}
```

- 장점: 빠르고, 똑같은 입력엔 똑같은 결과 (결정론적)
- 한계: 맥락**(context)**을 모른다. "warning이면 무조건 Major"라고만 안다.
  - 같은 warning이라도 인증(auth) 코드면 더 위험할 수 있다 → 모름
  - 이 팀이 과거에 똑같은 이슈로 사고가 났는지 → 모름
  - junior 개발자에겐 친절한 설명이 필요한지 → 모름

Task 4-b에서 할 것 **(context-aware)**

LLM은 위 맥락을 읽고 "이 도구는 **Major**라고 했는데, 과거 **PR #12**에서 같은 패턴이 보안 사고로 이어졌으니 **Critical**로 올린다" 같은 판단을 할 수 있다. 그러려면 LLM에게 맥락을 잘 떠먹여 줘야 한다 → 그게 프롬프트 엔지니어링.

> rule-based(CLI 도구)는 속도와 일관성, context-aware(LLM)는 맥락 이해. Dev2는 둘을 합친다: 도구가 1차 필터 → LLM이 2차 재판단.

🍕 1. 비유 — 신입 검사관 교육시키기

당신이 새로 들어온 신입 품질검사관에게 "불량품 등급(심각/보통/경미)을 매기라"고 시킨다고 상상하자. 그냥 "알아서 판단해"라고 하면 사람마다 기준이 들쭉날쭉하다. 그래서 4가지 장치를 준다:

기법	비유	효과
Few-shot	“이전 검사관들이 매긴 정답 예시 3개를 보여줌”	판단 기준을 맞춤(calibration)
Chain-of-Thought	“1번 무슨 결함? 2번 과거랑 비교? 3번 등급 결정 — 순서대로 적으라고 체크리스트를 줌”	건너뛰지 않고 차근차근 사고
RAG 주입	“예전에 같은 결함으로 리콜 났던 사고 기록을 책상 위에 올려줌”	우리 회사 맥락 반영
Structured Output	“결과는 반드시 이 양식(빈칸 채우기)에 적으라고 함”	다음 사람이 바로 읽음
Domain Rules	“Frontend/Backend 담당 매뉴얼을 따로 주고, 맡은 분야 규칙대로 판단하라고 함”	언어/프레임워크 특화 정확도↑

이 5개가 없으면? 신입은 자기 마음대로, 매번 다른 형식으로, 회사 사정 모르고 판단한다. 이 5개가 있으면? 베테랑처럼 일관되고, 우리 회사에 맞고, 기계가 읽을 수 있는 결과를 낸다.

📚 2. 개념 — 5가지 기법의 기술적 정의

(1) Few-shot Prompting

프롬프트 안에 입력→출력 예시 몇 개**(few)**를 넣어, LLM이 그 패턴을 모방해 답하게 하는 기법. 예시 0개 = zero-shot, 1개 = one-shot, 여러 개 = few-shot. - 핵심 원리: LLM은 “주어진 예시의 패턴을 이어서 완성”하려는 성질이 있다. 정답 예시를 보면 그 기준에 맞춰 답을 낸다(=calibration, 기준 보정). - 보통 3~5개가 적당. 너무 많으면 토큰 낭비 + 한쪽으로 편향(bias).

(2) Chain-of-Thought (CoT)

LLM에게 최종 답만 말하지 말고, 추론 과정을 단계별로 풀어내라고 지시하는 기법. - 핵심 원리: LLM은 “한 번에 정답”보다 “중간 사고를 거치며” 답할 때 정확도가 올라간다. (사람이 암산보다 종이에 적으며 풀 때 덜 틀리는 것과 같다) - “단계별로 생각해봐(Let’s think step by step)” 또는 번호 매긴 분석 단계를 제시하는 방식.

(3) RAG Context 주입 (Retrieval-Augmented Generation)

외부 저장소(여기선 pgVector)에서 관련 정보를 검색**(Retrieval)**해 프롬프트에 끼워 넣어(Augmented), LLM이 그 정보를 근거로 답을 생성(Generation)하게 하는 기법. - 핵심 원리: LLM은 학습 시점 이후의 정보나 "우리 회사 내부 데이터"는 모른다. 검색해서 넣어주면 그걸 근거로 답한다. - Dev2에선: 현재 이슈와 의미적으로 유사한 과거 **PR** 이슈를 pgVector 코사인 유사도로 찾아 주입.

(4) Structured Output

LLM 응답을 자유 문장이 아니라 정해진 구조**(**주로 **JSON)**로 강제하는 기법. - 핵심 원리: 프로그램이 받아 쓰려면 형식이 일정해야 한다. "JSON으로만 응답"이라고 명시 + 키 이름/타입을 미리 지정. - 받은 뒤엔 반드시 검증**(validation)** 해야 한다 → `dev2/schema.py`의 Pydantic `Issue` 모델로 검증. - 현재 출력 필드 **(5**개**)**: `severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale` - `issue_type`: `"Bug" | "Vulnerability" | "Code Smell"` — 이슈 성격 분류 - `resolution_example`: `"A 말고 B 써라"` 형태의 직관적 한 줄 교정 코드

(5) Domain Rules (도메인별 규칙 주입**)**

분석 대상 언어/프레임워크에 따라 다른 룰셋을 프롬프트에 주입하는 기법. - 핵심 원리: `eval()` 사용은 Python에서 경고지만 JavaScript에서는 XSS의 직접 원인이다. 같은 이슈도 도메인마다 severity가 달라진다. 파일 확장자와 `tech_stack`으로 FE/BE를 자동 판별 후 해당 규칙 섹션만 주입한다. - **Frontend Rules** (`_FRONTEND_RULES`): `eval()`, `dangerouslySetInnerHTML`, `var` 선언 등 JS/TS/React 특화 - **Backend Rules** (`_BACKEND_RULES`): SQL f-string, 하드코딩 API 키, `== None`, N+1 쿼리, `with` 구문 누락 등 Python/FastAPI 특화 - 도메인 감지 우선순위: 파일 확장자(`.js/.ts/.jsx/.tsx`) → `tech_stack` 리스트 (react/vue/svelte 포함 여부)

🧐 3. 왜 이 프로젝트에서 각 기법이 필요한가

기법	이 프로젝트에서의 구체적 이유
Few-shot	"SQL 미처리=Critical, 복잡도15=Major, 네이밍=Minor"처럼 **Dev2**의 **severity** 기준을 **LLM**에 학습시킨다. 안 넣으면 LLM이 자기 멋대로 등급을 매겨 도구 결과와 충돌.
CoT	severity는 단순 분류가 아니라 "패턴 파악→과거 비교→등급→레벨별 조언→한 줄 교정"의 **5**단계 판단이다. 단계를 강제해야 LLM이 과거 사례 비교를 건너뛰지 않는다.
RAG 주입	LLM은 우리 팀 과거 **PR** 이력을 모른다. pgVector로 찾은 `similar_cases`를 넣어줘야 "예전에 이걸로 사고났음 → 등급 상향" 같은 팀 맞춤 판단이 가능. CLAUDE.md §4 Dev1의 KMS 검색 결과를 활용하는 지점.
Structured Output	LLM 출력은 다음 단계(Dev4 평가, `dev2/schema.py` `Issue` 모델)로 넘어간다. JSON이 깨지면 파이프라인 전체가 멈춘다. CLAUDE.md §12 "LLM 응답이 잘못된 JSON" 처리 규칙과 직결. 현재 5개 필드(`severity`, `issue_type`, `suggestion`, `resolution_example`, `rationale`).
Domain Rules	Python `eval()`과 JS `eval()`은 위험도가 다르다. 파일 확장자+`tech_stack`으로 FE/BE를 자동 판별해 언어별 규칙을 동적으로 주입, 도메인 특화 정확도를 높인다.

한 줄 요약: **CLI** 도구는 **"규칙"**만 안다. **LLM**은 **"맥락"**을 더해야 가치가 생긴다. 그 맥락을 떠먹여 주는 5가지 숟가락이 이 5기법이다.

💻 4. 실제 코드 분석 (`dev2/prompts/severity_judge.py`)

전체 구조

이 함수는 LLM을 호출하지 않는다. "완성된 프롬프트 문자열"만 만들어 반환한다. (LLM 호출은 다른 노드에서, 개발 중엔 `USE_MOCK_LLM=true`로 Mock 처리 — CLAUDE.md §6)

부분별 역할

(A) Few-shot 블록 — 판단 기준 보정

```
FEW_SHOT_EXAMPLES = """
[판단 예시]
예시 1) SQL 파라미터 미처리 → Critical (입력값 직접 쿼리 삽입, 즉시 수정)
예시 2) 순환복잡도 15 → Major (리팩토링 권고, 테스트 커버리지 확인)
```

```
예시 3) 변수명 snake_case 위반 → Minor (코드 스멜, 다음 PR에서 수정)
"""
```

- 모듈 상단 상수로 분리 → 재사용 + 토큰 절약(매번 새로 안 만듦).
- 3개 예시가 각각 Critical/Major/Minor를 하나씩 → 3등급 기준을 골고루 보여줌(편향 방지).

(B) RAG context 조립 — 검색 결과를 문자열로

```
cases_text = "\n".join(
    f"- PR #{c['pr_id']} (유사도 {c['similarity']}): {c.get('message','')}"
    for c in similar_cases
) if similar_cases else "유사 사례 없음"
```

- `similar_cases`는 pgVector(`dev2/vector_store/`) 코사인 유사도 검색 결과.
- `c.get('message','')` → 키가 없어도 안 터지게 방어(빈 문자열 기본값).
- 빈 리스트 처리: 검색 0건이면 "유사 사례 없음"으로 대체 → CLAUDE.md §12 "pgVector 검색 결과 0건" 규칙 준수. (예외를 던지지 않고 진행)

(C) CoT 블록 — 단계별 사고 유도

```
[분석 단계 — Chain-of-Thought]
1단계) 현재 이슈의 패턴이 무엇인가? (SQL 인젝션? 복잡도? 코드 스멜?)
2단계) 과거 유사 사례와 비교하면 어떤 차이가 있나?
3단계) 최종 severity(Critical|Major|Minor)와 issue_type(Bug|Vulnerability|Code
Smell)을 결정하라
4단계) {developer_level} 수준의 개발자에게 맞춤 개선 방법을 제시하라
5단계) "A 말고 B 써라" 형태의 한 줄 resolution_example을 작성하라
```

- 번호 매긴 **5**단계로 LLM이 추론을 건너뛰지 못하게 함 (4단계 → 5단계로 확장).
- 3단계에 `issue_type` 결정 추가 → Bug/Vulnerability/Code Smell 분류 강제.
- 5단계에 `resolution_example` 작성 → "A 말고 B 써라" 형태의 한 줄 교정 코드.
- 4번에 `developer_level` 주입 → junior면 친절히, senior면 간결히 (CLAUDE.md §5-C 개인화 피드백).

(D) Structured Output 지시 — **JSON** 강제

```
JSON으로만 응답:
{{"severity": "Critical|Major|Minor", "issue_type": "Bug|Vulnerability|Code
Smell", "suggestion": "구체적 수정 방법 (코드 예시 포함)", "resolution_example": "A 말
고 B 써라 형태의 한 줄 교정 코드", "rationale": "판단 근거 한 줄"}}
```

- `{{ }}` 더블 중괄호 → f-string에서 리터럴 중괄호 출력 (`{ }` 한 개면 변수 치환으로 오인돼 에러).
- `severity` 값을 `Critical|Major|Minor`로 제한 → 받은 뒤 `dev2/schema.py`의 `Issue.severity` Literal과 매칭.
- `issue_type` 추가: `Bug|Vulnerability|Code Smell` — 이슈 성격을 3가지로 분류.

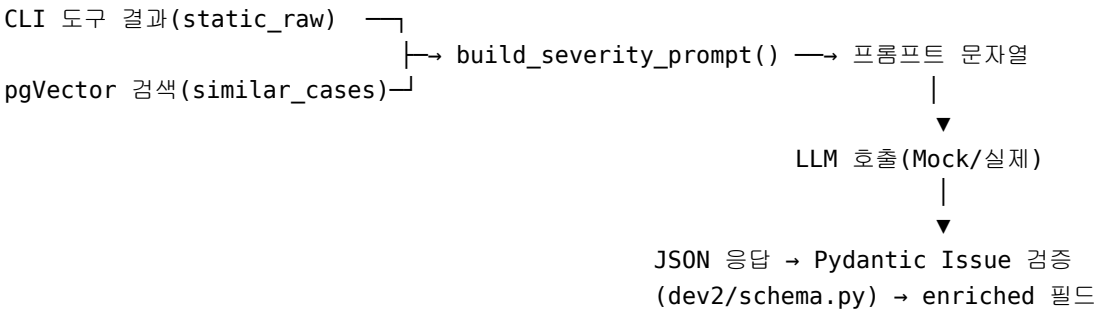
- `resolution_example` 추가: "A 말고 B 써라" 한 줄 교정 코드 → 개발자가 즉시 적용 가능한 구체적 안내.
- `rationale`(판단 근거)을 요구 → CoT의 결론을 명시적으로 받아 LangSmith 추적/디버깅에 활용.

(E) Domain Rules 주입 — 도메인별 규칙 선택

```
# 파일 확장자 + tech_stack으로 FE/BE 자동 판별
ext = file_name.rsplit(".", 1)[-1].lower() if "." in file_name else ""
fe_exts = {"js", "jsx", "ts", "tsx", "vue", "svelte"}
fe_stacks = {"javascript", "typescript", "react", "vue", "svelte", "nextjs"}
is_frontend = ext in fe_exts or bool(fe_stacks & {s.lower() for s in tech_stack})
domain_rules = _FRONTEND_RULES if is_frontend else _BACKEND_RULES
```

- 파일 확장자 우선 → `tech_stack`으로 보완 (확장자 없는 경우 대비).
- `_FRONTEND_RULES`: `eval()`, `dangerouslySetInnerHTML`, `var` 선언 등 JS/TS/React 특화 위험 패턴.
- `_BACKEND_RULES`: SQL f-string 조합, 하드코딩 API 키, `== None`, N+1 쿼리, `with` 구문 누락 등 Python/FastAPI 특화.
- 왜 따로 주입하나: `eval()`은 Python에서 경고 수준이지만 JS에서는 XSS 직접 원인(Critical). 같은 이슈도 도메인마다 severity가 다르다.

데이터 흐름 한눈에



3rd-Project-sub2 프로젝트에서

파일: `dev2/prompts/severity_judge.py` 역할: Task 3의 CLI 도구(rule-based)가 1차로 매긴 severity를, LLM이 과거 사례까지 보고 2차 재판단하도록 만드는 프롬프트 빌더. 출력은 `Dev2State.enriched`("pgVector 유사 이슈 + LLM 판단 결과")에 담겨, 최종적으로 `Issue` 모델로 검증돼 Output Contract A(`dev2/schema.py` `AgentOutput`)로 Dev4에 전달된다.

연결 관계: - 입력 `similar_cases` ← `dev2/vector_store/` (pgVector 코사인 검색) - 출력 JSON → `dev2/schema.py` `Issue` 모델로 검증 - JSON 파싱 실패 시 → CLAUDE.md §12: 재시도 1회 후 `{"issues": [], "error": "parse_failed"}`

⚠️ 흔한 실수 / 주의점

1. **f-string** 중괄호 — JSON 예시의 `{ }`는 반드시 `{{ }}`로. 한 개면 `KeyError` 또는 변수 치환 오류.
2. **JSON**을 믿지 마라 — LLM이 `json ...` 코드블록으로 감싸거나 설명을 덧붙이는 경우가 있다. 받은 뒤 반드시 파싱+Pydantic 검증, 실패 시 §12 규칙대로 재시도.
3. **Few-shot** 편향 — 예시를 Critical만 3개 넣으면 LLM이 전부 Critical로 매긴다. 등급을 골고루.
4. **RAG 0**건 미처리 — `similar_cases`가 빈 리스트일 때 "유사 사례 없음"으로 안 바꾸면 프롬프트에 어색한 빈칸이 생긴다.
5. 프롬프트 안에서 **LLM** 호출 금지 — 이 함수는 문자열만 만든다. 호출은 노드 책임(테스트 용이성↑).

복습 퀴즈 (3문항)

1. **(이해)** CLI 도구의 severity 매핑(rule-based)과 LLM 재판단(context-aware)의 가장 큰 차이는 무엇이며, Dev2는 왜 둘을 같이 쓰는가?
2. **(적용)** `similar_cases`가 빈 리스트로 들어왔다. 코드의 어느 부분이 이를 처리하며, 만약 그 처리가 없다면 프롬프트와 파이프라인에 각각 무슨 문제가 생기는가? (힌트: CLAUDE.md §12)
3. **(비교)** Few-shot과 RAG 주입은 둘 다 "프롬프트에 예시/정보를 넣는다"는 점이 비슷하다. 두 기법이 무엇을 넣는지, 왜 넣는지가 어떻게 다른가?

연관 개념

- **pgVector** / 코사인 유사도 → `docs/learning/vector/vectordb.md` (similar_cases를 만드는 검색)
- 정적 분석 도구 → `docs/learning/ai/static_analysis_tools.md` (재판단 대상인 1차 severity 생성)
- **Pydantic** 검증 → `dev2/schema.py` `Issue` 모델 (Structured Output을 받아 검증)
- **LangSmith** 추적 → CLAUDE.md §3 (rationale 필드가 디버깅에 활용됨)